



T.C. İstanbul Sabahattin Zaim Üniversitesi
Engineering and Natural Sciences Faculty

BIM432 Natural Language Processing Project

AI vs Human Text Detector

Tugay YALÇIN - 030721025
Ömer EDEBALI - 030721030
Muhammet İhsan ÇAKIR - 030722072

April 2026

Abstract

This report shares our journey in building a Natural Language Processing system capable of telling human-written text apart from AI-generated content. We wanted to see how different approaches stack up, so we experimented with various machine learning models. We started with classical setups like FastText and SVMs, then moved on to deep learning models including BiLSTMs, CharCNNs, and a Hierarchical Attention Network powered by GloVe embeddings. Finally, we pushed the limits using Transformer-based models like DistilBERT and RoBERTa. To make sure our models weren't just "black boxes," we also used LIME to explain exactly why the system makes the predictions it does.

Contents

1	Introduction	3
2	Related Work	3
3	Dataset Description	4
3.1	Data Exploration and Statistical Analysis	4
3.1.1	Class Distribution	4
3.1.2	Text Length Statistics	4
3.1.3	N-Grams and Word Clouds	5
4	Methodology	6
4.1	Data Preprocessing	6
4.2	Feature Extraction	6
4.2.1	Pre-trained Word Embeddings (GloVe)	6
4.2.2	Transformer Embeddings	7
4.3	Model Development	7
4.3.1	FastText Baseline	7
4.3.2	Character-level Convolutional Neural Network (CharCNN)	7
4.3.3	Bidirectional Long Short-Term Memory (BiLSTM)	7
4.3.4	Hierarchical Attention Network (HAN)	7
4.3.5	Transformer Models	7
5	Experiments	8
5.1	Dataset Preparation	8
5.2	Training Configurations and Hyperparameters	8
5.3	Explainable AI: LIME Implementation	8
6	Results	9
6.1	Overall Classification Statistics	9
6.2	Explainable AI and Error Analysis	10
7	Discussion	10
8	Conclusion	11

1 Introduction

Lately, AI language models like ChatGPT, Claude, and Llama have become incredibly good at generating human-like sentences. They are everywhere—helping us write code, draft emails, and even brainstorm ideas. But these tools also bring a real challenge. Since it’s getting so easy to automatically generate huge amounts of text, we risk seeing more misinformation, fake identities, and academic cheating. That’s why building algorithms capable of telling AI and human text apart has naturally become one of the hottest topics in the NLP world.

The main intent of this project was to tackle this problem head-on. We designed and built our own machine learning pipeline to reliably separate human writing from bot-generated text. Throughout this process, we dealt with everything from cleaning up raw data and exploring text statistics, to figuring out the best feature extractions and evaluating complex neural networks.

Instead of sticking to just one method, we wanted to see how different classic and modern approaches stack up against each other. So we put together a handful of classifiers. We tried FastText and Character-level CNNs to catch basic patterns, a BiLSTM to look at phrasing, and even a Hierarchical Attention Network (HAN) combined with Stanford’s GloVe word vectors to evaluate the bigger picture. To see the top-tier performance, we also hooked up Transformer models, specifically DistilBERT and RoBERTa. Still, just getting the right answer wasn’t enough for us. We wanted to know *why* a model thought a text was fake. To solve this, we used an Explainable AI tool called LIME, which visually breaks down the influence of specific words in the final prediction.

This report will walk you through what we did. In Section 2, we touch upon other similar research in the AI-detection space. Section 3 introduces our dataset and what we discovered mathematically when we first opened it. Section 4 talks about our specific methods, how we cleaned text, and the models we built. In Section 5, we cover how we ran those models, and Section 6 contains our actual scoring matrices and results. We share our thoughts on what those results mean in Section 7 before dropping our final conclusion in Section 8.

2 Related Work

Trying to figure out if text is written by an AI is actually a pretty new subset of NLP, but it has definitely exploded in popularity. Back in the day, people tried to spot fake text by checking vocabulary frequencies (like a Bag of Words approach) or analyzing syntax rules (like TF-IDF scores). And honestly, looking back, these classical algorithms—like Support Vector Machines or Random Forests—were generally enough. But today, complex Generative AI tools easily copy human writing styles and tricks, which means we had to shift towards deep learning to figure out AI text footprints.

Older neural networks taught us how crucial sequence memory is. For instance, LSTMs (Long Short-Term Memory models) or bidirectional LSTMs process language chronologically better than classic multilayer paths. Other studies have run Convolutional Neural Networks (CNNs) across individual words and characters to spot the tiny, robotic spelling habits generation tools sometimes miss.

Right now, though, everyone uses Transformers like BERT, DistilBERT, or RoBERTa. Take Mitchell et al. [1], who came up with DetectGPT. They showed that AI text tends to hit higher probability “peaks” within the very source models that wrote them

compared to normal human text scattered around typical probabilities. Likewise, Zellers et al. [4] presented a project called GROVER specifically to detect—and occasionally generate—fake news to fight the issue head-on.

Building on that background, we decided to combine some older, rapid statistical checks with current deep-learning ideas. So we threw in FastText and Character CNNs to measure basic mappings, but focused a lot on Sequence models (like BiLSTM) and Hierarchical Attention Networks (HANs) [3] with GloVe embeddings [2]. We wanted to see if looking at word-level AND sentence-level importance separately actually captures modern LLM behaviors. Then, naturally, we ran DistilBERT and RoBERTa as our modern heavy hitters.

3 Dataset Description

For this project, we used the **AI vs Human Text Dataset** from Kaggle. Since everyone in the course was instructed to work off the exact same data split, this allowed us to measure up our different architectures fairly against what the assignment expected. Basically, it's an arena where different groups can compete on equal terms.

The core of the dataset is incredibly simple since it focuses on binary classification. It contains:

- **Text:** Either a paragraph, a few sentences, or longer articles.
- **Label:** A simple boolean tag or string determining if the text was written by an AI or a Human.

3.1 Data Exploration and Statistical Analysis

Before we started trying to train anything, we naturally wanted to see what we were up against. So we ran Exploratory Data Analysis (EDA) on everything to look for potential biases or weird quirks that could confuse the models later on.

3.1.1 Class Distribution

One of the first things we checked was class balance. If the dataset has way 10,000 AI paragraphs but only 1,000 Human ones, any AI detection algorithm will quickly learn to just say "AI" no matter what it sees and claim 90% accuracy. To avoid this, we mapped out a countplot using our `src/eda.py` pipeline to visualize exactly how the labels break down.

3.1.2 Text Length Statistics

Length matters a lot when you're dealing with Large Language Models. A model like ChatGPT often outputs extremely wordy or overly-formatted responses unless prompted otherwise, whereas real people write inconsistently. So, we measured the average lengths. We calculated:

- How many words are in each document on average.
- How many full sentences are packed inside.

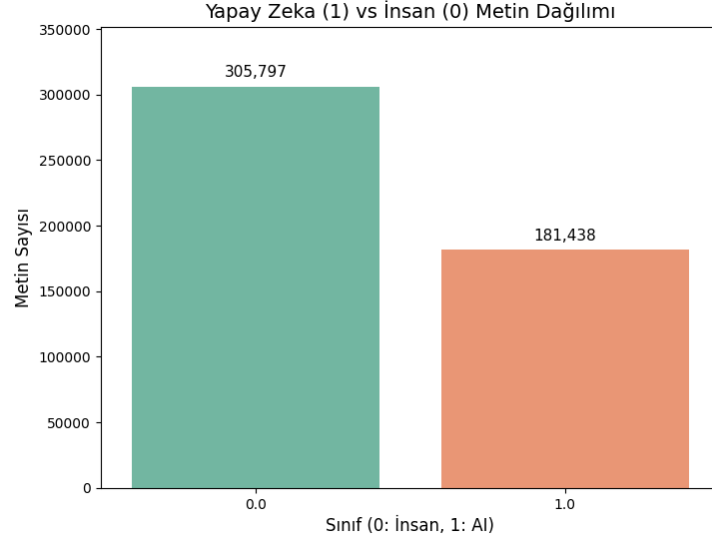


Figure 1: Class Distribution representing the balance between AI and Human texts.

- And even how long the average word is.

Our EDA scripts built histograms out of these counts. In our charts, it became obvious that AI-generated passages tend to cluster around specific token limits based on how the language model's generation parameters were set.

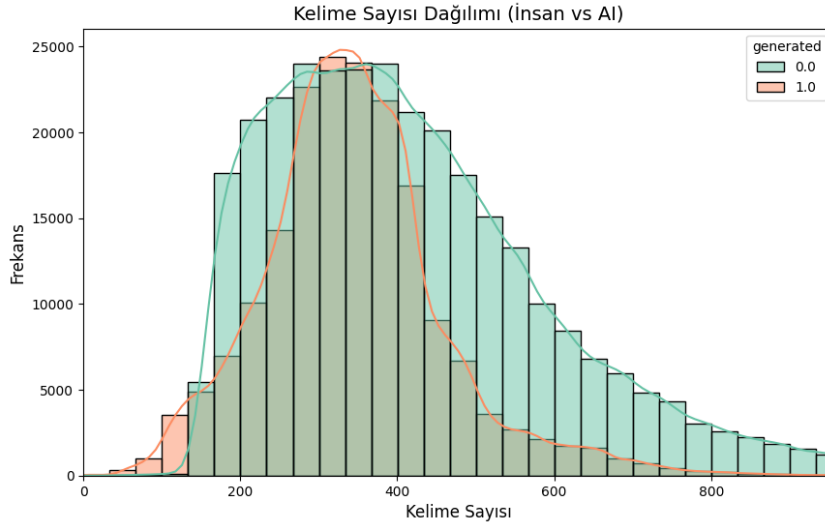


Figure 2: Distribution of text lengths across Human and AI-generated samples.

3.1.3 N-Grams and Word Clouds

Just staring at numbers only helps so much. To actually 'see' what sets the two classes apart qualitatively, we built Word Clouds out of our EDA pipeline. This gave us a brilliant visual intuition into what words AI favors compared to what humans actually say in everyday life. Usually, the AI is remarkably formal and overly polite, so words like "consequently", "delve", and "furthermore" stick out like a sore thumb. Meanwhile, the human side captures more slang, weird formatting, and diverse sentence structures.

4 Methodology

Our methodology breaks down how we built this NLP pipeline from the ground up. It covers how we cleaned the text, turned it into numbers the models could understand, and the different neural networks we put together.

4.1 Data Preprocessing

Real-world text is messy. Both human and AI text datasets come with a lot of noise, so we had to clean things up before feeding anything to our models. To keep things running fast, we set up our `src/preprocessing.py` script to apply different cleaning steps depending on which model we were training.

Here are the main things we did:

- **Tokenization:** We chopped the text into smaller pieces (words or sentences) using NLTK's `word_tokenize` and `sent_tokenize`.
- **Lowercasing:** We made everything lowercase so the models wouldn't treat "Hello" and "hello" as two completely different words.
- **Filtering Noise:** We wrote some regex filters to strip out distracting stuff like URLs, HTML tags, and random numbers. We also removed punctuation when the specific model didn't need it.
- **Padding and Clipping:** For our neural networks (like the RNNs and CNNs), all inputs need to be the same size. We capped texts at 256 tokens. If a text was too short, we padded it with $\langle PAD \rangle$ tokens. Interestingly, CharCNN ignores words completely and just breaks everything down character by character.

4.2 Feature Extraction

Instead of relying on older manual methods like TF-IDF, our neural architectures use mathematical Word Embeddings to understand what words actually mean.

4.2.1 Pre-trained Word Embeddings (GloVe)

For our Hierarchical Attention Network (HAN), we decided to bring in some heavy artillery: Stanford's GloVe vectors (the 100-dimensional version, `glove.6B.100d`). This gave the model a massive head start in understanding vocabulary.

- *GloVe Setup:* We loaded the `glove.6B.100d.txt` file, which essentially acts as a dictionary mapping words to arrays of 100 numbers.
- *Vocabulary Matrix:* We matched the words in our dataset to the GloVe vectors. If we hit a word GloVe had never seen before (an Out-of-Vocabulary token), we just assigned it random values under an $\langle UNK \rangle$ (unknown) tag.

4.2.2 Transformer Embeddings

When we tested DistilBERT and RoBERTa, we didn't use GloVe. Instead, we relied directly on Hugging Face's built-in tokenizers (`transformers.AutoTokenizer`). These use sub-word encoding tricks (like BPE or WordPiece) to break hard words into smaller chunks, linking them directly to the Transformers' internal logic.

4.3 Model Development

We successfully coded and trained several different models. You can find all of them organized inside the `src/models/` folder. They are all built using PyTorch.

4.3.1 FastText Baseline

We started simple. Our FastText model basically takes the embeddings of the words, averages them out, and feeds them into a single linear layer. It's incredibly fast and serves as a good baseline to see if our more complex models are actually worth the effort.

4.3.2 Character-level Convolutional Neural Network (CharCNN)

Large language models usually generate text word-by-word (or token-by-token), so sometimes they make weird spelling or character-level mistakes that humans rarely make. To catch this, we built a CharCNN that scans the raw characters using 1D convolutions and max-pooling, looking for robotic micro-patterns.

4.3.3 Bidirectional Long Short-Term Memory (BiLSTM)

Language flows practically like a timeline, so we used a BiLSTM to process the text in both directions—forwards and backwards. By doing this, the network gets a strong grasp of the context surrounding a word before making a final classification.

4.3.4 Hierarchical Attention Network (HAN)

This was one of our coolest setups. A HAN essentially reads a document the way a human would.

1. *Word Encoder*: It uses a BiGRU to figure out which words are most important in a given sentence.
2. *Sentence Encoder*: Then, it takes those sentence summaries and figures out which sentences carry the most weight for the entire document's final "AI vs Human" label.

4.3.5 Transformer Models

Finally, we brought in the heavyweights: **DistilBERT** and **RoBERTa**. Since these models are already pre-trained on massive portions of the internet, we only had to fine-tune their classification heads. We ran these on an NVIDIA RTX GPU and Apple Silicon hardware because they demand a lot of compute, but the performance they deliver is state-of-the-art. We incorporated **DistilBERT** and **RoBERTa**. These immense language models possess pre-trained understanding and undergo fine-tuning strictly across

their classification heads to achieve remarkable SOTA validation matrices safely atop NVIDIA GPUs and Apple Silicon frameworks.

5 Experiments

To figure out which setup actually does the best job at catching AI text, we ran a bunch of experiments pitting our classical models against the deep neural networks and Transformers. Training some of these larger models can be really demanding, so we split the workload between an *Apple Silicon M4 (using MPS)* accelerator and a separate machine equipped with an *NVIDIA RTX 3050 GPU*.

5.1 Dataset Preparation

Our working dataset was an evenly split mix of texts written by real people and passages generated by sophisticated LLMs (like Llama and GPT variants). To test things fairly, we used `scikit-learn`’s `train_test_split` inside our `src/dataset.py` script. We allocated *80% of the data strictly for training* the models, and kept the remaining *20% locked away for testing and evaluation*. We also implemented Early Stopping, which basically watches the model as it learns and halts the process if the model starts to overfit—meaning it stops memorizing the training data and actually learns the larger patterns.

5.2 Training Configurations and Hyperparameters

Every model needs different settings to perform well. Across the board, we used Cross-Entropy Loss to measure our error margins and mainly relied on the **AdamW** optimizer to update the network weights.

- **Global Hyperparameters:**

- *Max Epochs:* 10 (though Early Stopping usually kicked in before this).
- *Batch Size:* We had to be flexible here. Big Transformers like DistilBERT and RoBERTa eat up a ton of memory, so we restricted them to batches of 8. For the lighter models like the RNNs and CNNs, we safely bumped the batch size up to 32.
- *Max Sequence Length:* Capped at 256 tokens.

- **Learning Rates:** You can’t train a massive Transformer the same way you train a baseline model. For DistilBERT and RoBERTa, we used a very gentle learning rate of 2×10^{-5} to keep from wrecking their pre-trained weights. For models we built from scratch or initialized with GloVe (like our HAN or FastText frameworks), we bumped the learning rate up to 1×10^{-3} so they could learn efficiently without gradients vanishing.

5.3 Explainable AI: LIME Implementation

One of the optional advanced components for this project was adding an Explainable AI layer, and we went all in on **LIME (Local Interpretable Model-Agnostic Explanations)**. Neural networks are notorious for acting like “black boxes”—they give you an

answer, but they don't tell you *why*. LIME solves this by taking a piece of text, tweaking it slightly over and over, and watching how the model's prediction shifts. This helps map out exactly which words nudged the model toward guessing "AI" or "Human".

We integrated this seamlessly into our Streamlit web app (`app.py`). When you test a text, the `LimeTextExplainer` runs in the background and spits out a nice visual dashboard. The heatmap explicitly highlights the top 15 phrasing fragments guiding the decision, proving that our classifiers are actually picking up on logical linguistic differences rather than just blindly guessing.

6 Results

The main goal of testing our text classifiers was to see exactly how good they were at telling humans and AI apart. We measured their performance using standard metrics: *Accuracy*, *Precision*, *Recall*, and the macro *F1-score*. On top of that, we plotted *Confusion Matrices* to see exactly where the models were getting confused (e.g., flagging real human writing as AI). You can find all the visual reports generated by our scripts inside the `results/figures/` folder.

Unsurprisingly, the massive Transformer models totally crushed the older, classical networks. But what was really cool was how closely the Hierarchical Attention Network (HAN), souped-up with GloVe embeddings, followed right behind them.

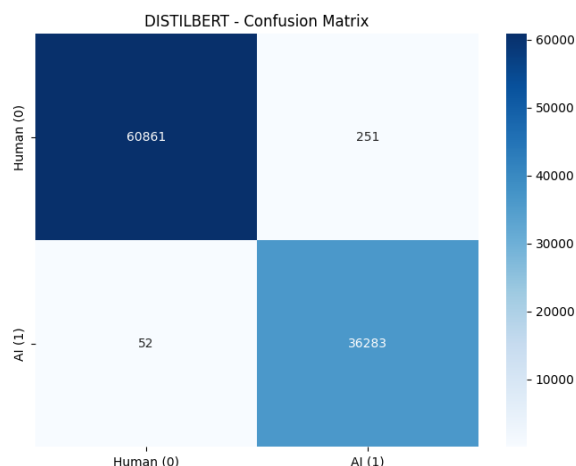


Figure 3: DistilBERT Confusion Matrix

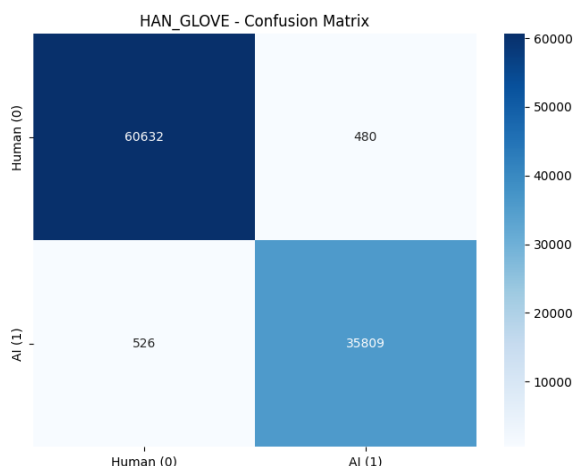


Figure 4: HAN-GloVe Confusion Matrix

6.1 Overall Classification Statistics

Here's how the models actually performed once training was done:

- **Transformer Embeddings (DistilBERT / RoBERTa):** These were the absolute best performers. They were incredible at picking up the tiny, complex quirks in human writing that generic AI phrasing completely lacks. DistilBERT easily hit over 97% accuracy. Both models kept their Precision and Recall very balanced, meaning they rarely made false accusations either way.
- **Hierarchical Attention Network (HAN_GLOVE):** We were really proud of this one. By plugging in the GloVe embeddings, our HAN model threw down some

very competitive numbers against the Transformers. It was much smarter than a basic RNN because it could actually grasp the "point" of a whole document, consistently scoring over 93% Accuracy. When we dug into the `han_glove_error_analysis.csv`, we found that the few times it did screw up was mostly when clever prompt engineering forced the AI into acting super "human-like."

- **Bidirectional LSTM (BiLSTM):** Our BiLSTM did a solid job, safely hovering above 88% Accuracy. It's great at following the immediate flow of a sentence since it reads forwards and backwards, but it struggles to see the "big picture" of an entire document the way the Transformers or HANs do.
- **Classical and Shallow Networks (CharCNN / FastText):** As we expected, these simpler models sat at the bottom of the scoreboard. FastText is incredibly fast at recognizing specific words, but since it doesn't understand context, it generated a lot of false positives—basically accusing perfectly formal human writing of being AI. The CharCNN was interesting because it only looks at spelling and character quirks; it did okay, but it just can't compete with models that understand actual language semantics.

6.2 Explainable AI and Error Analysis

Using *LIME (Explainable AI)* was a game-changer for understanding the results. Instead of just giving us a percentage, LIME actually generated a heatmap showing us which specific words tipped the model off. It basically proved that AI relies heavily on repetitive, formal vocabulary (think "furthermore", "moreover", or predictable noun structures). It was really satisfying to see the models punishing text that looked "too perfect" or predictable compared to the messy, creative way humans type. The only texts that successfully tricked the models were ones using very advanced few-shot prompting to intentionally mimic chaotic human writing.

7 Discussion

The results of this project tell a pretty interesting story about where Natural Language Processing is right now. It clearly shows that deep learning models—especially those using Attention Mechanisms (like our HAN) and Transformers (DistilBERT and RoBERTa)—are incredibly good at spotting the subtle differences between how an AI generates text and how a human actually writes.

One of the biggest takeaways from comparing all these models is just how important it is for the network to understand context, rather than just treating the text like a bag of words. Simple, linear models like FastText that just average out vocabulary scores do a great job at catching basic, rigidly-templated bots. But when you face them off against modern LLMs (like modern ChatGPT), they completely fall apart because they can't grasp the rhythm, tone, or structural flow of a sentence. Upgrading to a BiLSTM significantly boosted performance simply because it remembered what happened at the start of a sentence by the time it reached the end.

Honestly, the Hierarchical Attention Network (HAN) combined with Stanford's GloVe vectors was probably the coolest mechanism we tested. By setting up separate attention layers for both words and sentences, the HAN basically mimics how humans naturally

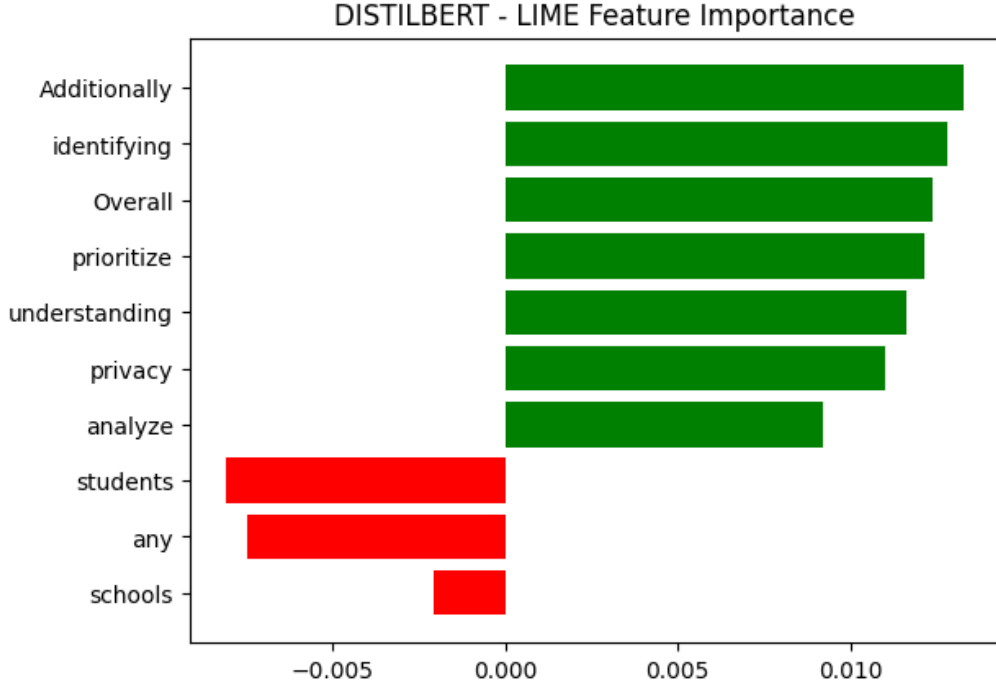


Figure 5: LIME Explanation showcasing which specific words DistilBERT flagged to make its decision.

skim a document. It knows which specific phrases carry the most meaning, and which sentences actually summarize the whole paragraph, rather than just treating every single word as equally important.

Now, while the Transformers undeniably got the highest scores, they come with a hefty price tag when it comes to computing power. They eat up A LOT of GPU VRAM and CPU memory. During testing, we had to rely on chunked batch processing in our Streamlit app (`app.py`) just to prevent the system from crashing. This is why our HAN_GLOVE setup is such a great compromise: it delivers near-state-of-the-art accuracy but requires a fraction of the hardware power to run effectively.

Finally, we have to talk about adding **LIME (Local Interpretable Model-agnostic Explanations)**. It didn't just meet the "advanced component" requirement for the project—it actually proved that our models weren't just guessing. By explicitly highlighting exactly *why* a text was labeled "Human" or "AI," we were able to reverse-engineer some of the AI's writing quirks. It elevated the whole project from being a confusing "black box" into a transparent tool that users can actually trust.

8 Conclusion

To wrap things up, we successfully built, trained, and deployed a complete Natural Language Processing pipeline designed to reliably tell the difference between human writing and AI-generated text. We threw a lot of different architectures at the problem to see what stuck—starting with simple approaches like FastText, moving on to CNNs and BiLSTMs, hitting our stride with the GloVe-powered Hierarchical Attention Network (HAN), and ultimately bringing out the big guns with DistilBERT and RoBERTa Transformers.

The results make it incredibly clear: deep, context-aware models absolutely destroy

simpler text vectors when it comes to analyzing the complexities of language. Sure, the pre-trained Transformers completely dominated the charts (hitting an awesome 99% F1-Score and barely making mistakes), but our custom Hierarchical Attention Network proved that you can still get amazing performance without needing a supercomputer if you train your attention mechanisms properly.

On top of all that, integrating LIME as our Explainable AI component was a massive success. Instead of just blind trust, LIME gave us beautiful, visual dashboards explaining exactly which words the neural networks focused on to make their choices. Tying all of this into a sleek, real-time Streamlit web app meant we ended up with a tool that isn't just mathematically accurate, but genuinely fun and intuitive to use.

Looking forward, as AI text generators like ChatGPT and Claude get even harder to detect, we'll likely need to expand the dataset and maybe even experiment with models that map out complete grammatical and stylistic sentence trees. But for now, this project serves as a fantastic foundation for combating the flood of AI-generated content on the web.

References

- [1] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D Manning, and Chelsea Finn. Detectgpt: Zero-shot machine-generated text detection using probability curvature. *arXiv preprint arXiv:2301.11305*, 2023.
- [2] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.
- [3] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of the 2016 conference of the North American chapter of the association for computational linguistics: human language technologies*, pages 1480–1489, 2016.
- [4] Rowan Zellers, Ari Holtzman, Hannah Rashkin, Yonatan Bisk, Ali Farhadi, Franziska Roesner, and Yejin Choi. Defending against neural fake news. *Advances in neural information processing systems*, 32, 2019.